

UNIVERSIDAD NACIONAL AUTONOMA DE MEXICO

Facultad de Estudios Superiores Cuautitlán

Ing. Telecomunicaciones Sistemas y Electrónica

Carlos Yahir Montoya Nava

20 de Septiembre de 2026

Sistema Basado en Microprocesador Z80

Resumen

En este trabajo se diseña el hardware y software de un microprocesador Z80. El diseño de hardware se desarrolló en la plataforma KiCad, implementando un mapa de memoria estructurado mediante decodificación lógica que incluye 16 KB de memoria ROM iniciando en la dirección 0000h, 16 KB de memoria SRAM iniciando en F800h, y 6 puertos de entrada/salida mediante dos interfaces PPI 8255 iniciando en la dirección 40h. Para el software, se desarrolló un programa en lenguaje ensamblador, validado en el simulador Z80_workbench, valida estrictamente el ingreso exclusivo de letras y espacios (rechazando números y símbolos), y almacena dicha cadena para que después se imprima. Todo el control de versiones se gestionó mediante un repositorio en GitHub.

Introducción

Los microprocesadores constituyen el núcleo lógico y aritmético de los sistemas computacionales. El Z80, un procesador clásico de 8 bits de arquitectura CISC, es un componente ideal para el estudio y comprensión de los fundamentos de la arquitectura de computadoras gracias a la accesibilidad directa de sus buses de datos, direcciones y señales de control.

El objetivo del proyecto es aplicar los conocimientos de diseño lógico combinacional y programación a bajo nivel para estructurar una computadora

funcional desde cero. Esto implica el reto técnico de decodificar un mapa de memoria con rangos operativos estrictos utilizando decodificadores (74LS138) y compuertas lógicas (NAND, OR), asegurando que no existan colisiones en los buses. Asimismo, se busca lograr una integración perfecta entre el hardware diseñado y un software escrito en ensamblador capaz de controlar el flujo de información y validar datos de entrada.

Funcionamiento del sistema

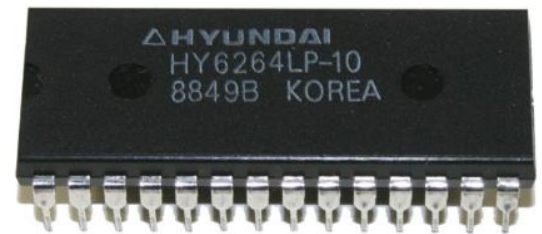
Microprocesador

- **Descripción:** Se utilizó el microprocesador Z84C00 (Z80 CMOS) en encapsulado DIP de 40 pines. Posee un bus de datos bidireccional de 8 bits y un bus de direcciones de 16 bits, lo que le otorga la capacidad de direccionar hasta 64 KB de memoria y
- 256 puertos de entrada/salida.
- **Señales a utilizar:**
 - **Buses:** A0-A15 (bus de direcciones) y D0-D7 (bus de datos).
 - **Control de estado:** MREQ (Memory Request) indica que el bus de direcciones tiene una dirección válida para memoria; IORQ (Input/Output Request) indica una dirección válida para un periférico.
 - **Transferencia:** RD (Lectura) y WR (Escritura).
- **Funcionamiento:** El Z80 opera bajo un ciclo continuo de búsqueda, decodificación y ejecución. Utiliza las combinaciones de las señales MREQ, IORQ, RD y WR para coordinar de forma síncrona qué dispositivo externo (memoria o puerto) debe escribir o leer información del bus de datos en un instante de tiempo específico.



Memorias

- **ROM (28C256):** Es una memoria EEPROM no volátil utilizada para almacenar el código ejecutable en lenguaje ensamblador del sistema. Aunque el chip tiene capacidad de 32 KB, el diseño direcciona específicamente 16 KB conectando los pines A0 a A13. Durante la operación normal del sistema, su pin de escritura (WE) se mantiene conectado a VCC para asegurar que el procesador solo pueda realizar ciclos de lectura.
- **SRAM (6264):** Son memorias de acceso aleatorio estáticas y volátiles. En este diseño se implementaron dos circuitos integrados 6264 (de 8 KB cada uno) para cumplir con el requerimiento de 16 KB de memoria de datos. Estas memorias permiten ciclos de lectura y escritura (OE y WE habilitados) y se utilizan para almacenar temporalmente los datos ingresados por el usuario, como la cadena de caracteres de entrada.



Interfaz para periféricos

- **PPI (8255):** La interfaz periférica programable 8255 es un controlador de entrada/salida diseñado para uso con microprocesadores. Cada chip proporciona 24 pines de E/S programables distribuidos en tres puertos de 8 bits (Puerto A, Puerto B y Puerto C). El diseño incorpora dos de estos integrados para cumplir con el requisito de 6 puertos en total. Para su direccionamiento, el 8255 utiliza los pines A0 y A1, mientras que su activación depende de la señal IORQ del microprocesador.

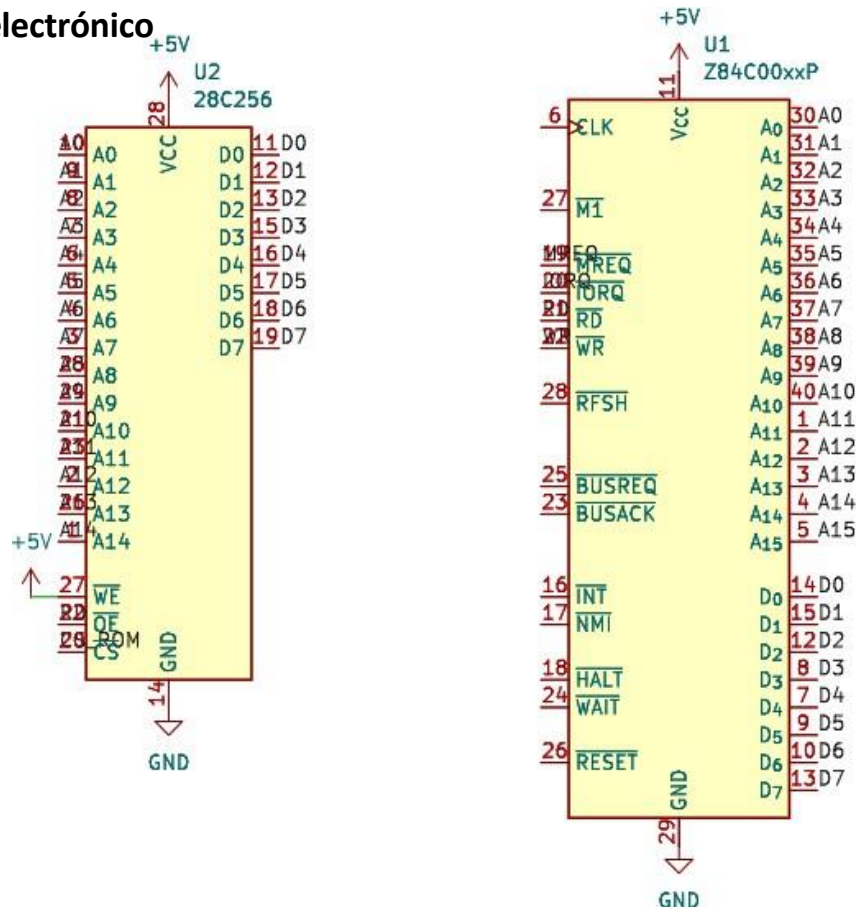


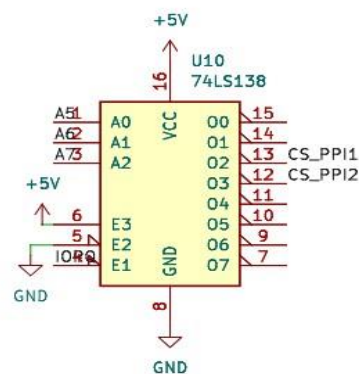
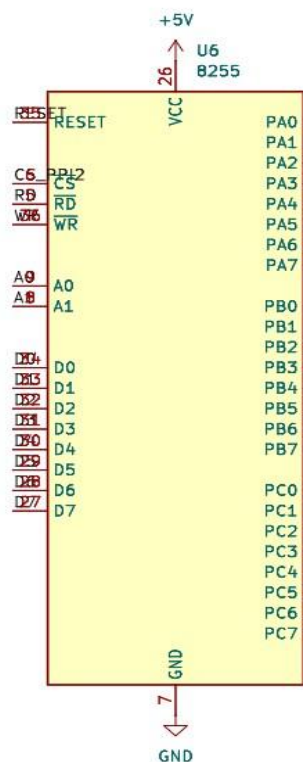
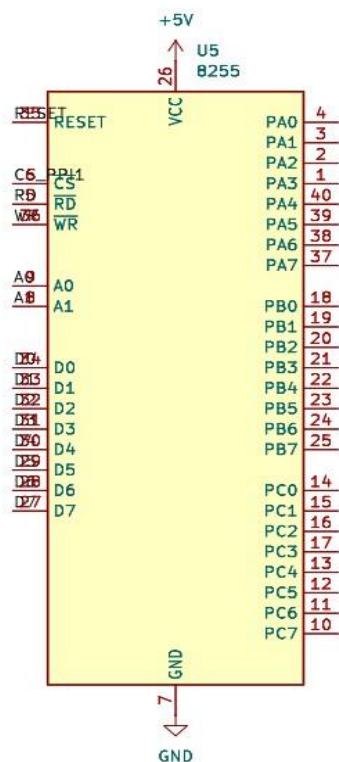
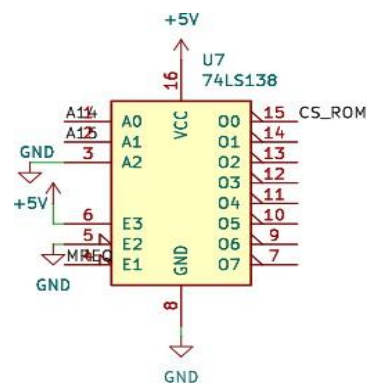
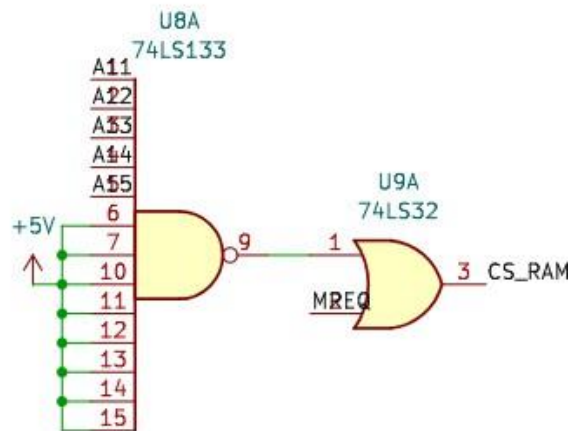
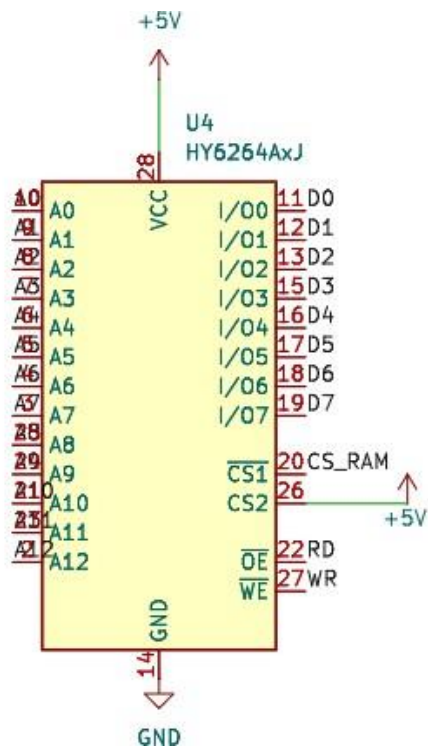
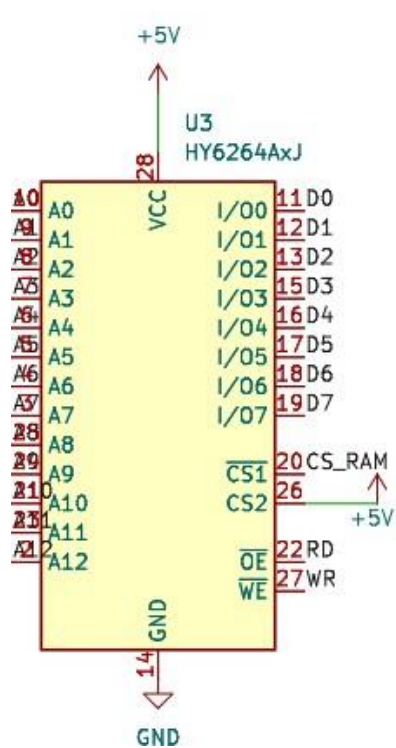
Diseño del sistema

El diseño del hardware se basó en el cumplimiento estricto del mapa de memoria:

1. **ROM (0000h a 3FFFh):** Se utilizó un decodificador 74LS138 habilitado por la señal MREQ. Sus entradas de selección evaluaron los pines A14 y A15 del bus de direcciones para activar el chip select de la EEPROM únicamente cuando ambos bits fuesen 0.
2. **RAM (F800h en adelante):** La dirección base F800h (1111 1000 0000 0000 en binario) se decodificó implementando una compuerta NAND de 5 entradas (analizando desde A11 hasta A15). La salida se validó en conjunto con MREQ mediante una compuerta OR para activar las SRAM 6264.
3. **Puertos (40h):** Se utilizó un segundo decodificador 74LS138 habilitado por IORQ. Al evaluar los pines A5, A6 y A7, la salida O2 (dirección 40h) activó el primer PPI 8255, y la salida O3 (dirección 60h) activó el segundo PPI.

Diagrama electrónico





Análisis de los resultados

El diseño lógico en KiCad salio correctamente, garantizando que el bus de datos no sufra de colisiones, ya que las señales lógicas de los decodificadores y compuertas aseguran la exclusión mutua; es decir, la ROM, la RAM y los puertos PPI jamás intentarán escribir en el bus al mismo tiempo.

Por el lado del software, la rutina en lenguaje ensamblador validó con éxito el ingreso de caracteres. Al utilizar operaciones de comparación (CP) y saltos condicionales (JR C, JR NC, JR Z), el programa filtró correctamente la tabla ASCII, permitiendo el registro exclusivo de códigos hexadecimales correspondientes a letras mayúsculas (41h-5Ah), letras minúsculas (61h-7Ah) y el espacio (20h), logrando el objetivo de entrada controlada planteado en el proyecto.

Conclusión

El desarrollo de este proyecto permitió comprender la relación que existe entre el diseño de hardware (decodificación de direcciones) y la ejecución de software a bajo nivel (ensamblador). Se comprobó que el microprocesador requiere un diseño electrónico preciso y libre de ambigüedades en sus señales de habilitación para funcionar de manera estable. Asimismo, la utilización de herramientas de diseño como KiCad, simuladores de código como Z80_workbench y plataformas de control de versiones como GitHub resultaron fundamentales para documentar, estructurar y asegurar la integridad de un proyecto de ingeniería.

Referencias

- Zilog, Inc. (2001). *Z80 Family CPU User Manual*.
- Atmel Corporation. *AT28C256 Datasheet - 256K (32K x 8) Paged Parallel EEPROM*.
- Intel Corporation. *8255A Programmable Peripheral Interface Datasheet*.
- Documentación oficial de KiCad EDA. Recuperado de docs.kicad.org.

Z80 · WORKBENCHv1.2

assembler + educational emulator for the Zilog Z80 microprocessor

by jitsajada@bhdyn.com

HELLO (OUT) ▾

ASSEMBLE

STEP

RUN

STOP

RESET

INT

50HZ INTERRUPT (LIKE THE ULA)

SPEED

SOURCE CODE · Z80 ASM145 BYTES

LOADprogram.z80

DOWNLOAD

BURN TO ROM: .BIN (RAW BINARY) ▾

DOWNLOAD ROM

ES_LETRA:

INC B

ES_ESPACIO:

LD (HL), A

INC HL

OUT (01h), A; Imprimir letra

oceptada

JR LEER_TECLA

ES_ERROR:

LD DE, MSG_ERR

CALL IMPRIMIR_STR

JR LEER_TECLA

FIN_LECTURA:

LD DE, MSG_RES

CALL IMPRIMIR_STR

Assembled successfully. 60 instructions · origin 0x0000

CPU · REGISTERS

PC0046

SPFFFF

A36

F00

BC0600

DE068A

HLF807

IX0094

IV0000

I00

R00

STEPS369

S

Z

H

PV

N

C

IFF1

HALT — PROGRAM STOPPEDIM

SHADOW REGISTERS (EXX / EX AF,AF')

MEMORY

FOLLOW PC: ON

00300110DC116F00CD46----0--F

00380010D4117900CD46----y--F

00400078CD4F00761AFE--x:0:v--

004800C8D3011018F7DE-----

005000FE0A3805060A0C---B---

005810F75779C630D301--WY:0--

00607AC630D301C94E6Fz-0---No

0068606272653A200020mbre:--

0070584552524F525D20[ERROR]

007800202020546F7461-- - Tota

00806020606574726173l letras

00883A2000436172606F: - Carlo

SCREEN · MEMORY-MAPPED

Same layout as the ZX Spectrum: 0x4000-0x57FF
pixel bitmap (256x192 px) - 0x5800-0x5AFF color
attributes (32x24 cells):
ink/paper/bright/flash). Write there with LD
(nn),A and it draws instantly.

CLEAR SCREEN

VIEW IN MEMORY

CONSOLE · OUT (1),A

Nombre: Carlos [ERROR] -
Total letras: 06

EXECUTION TRACE

yahirmontnv / Proyecto-Microprocesadores ▾

Type 17 to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

Proyecto-MicroprocesadoresPublic

Pin

Watch 0 ▾

Fork 0

Star 0 ▾

main ▾

1 Branch

0 Tags

Go to file

Add file ▾

<> Code ▾

yahirmontnvDelete hardware/temporal.txt

9fd61f · 4 days ago

8 Commits

documento

Delete documento/temporal.txt

4 days ago

hardware

Delete hardware/temporal.txt

4 days ago

software

Create codigo.z80

4 days ago

README.md

Initial commit

4 days ago

README

Proyecto-Microprocesadores

Proyecto Microprocesadores

About

Proyecto Microprocesadores

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

yahirmontnv